COMP5047: Applied Software Engineering

Coursework 1

GitHub Link: https://github.com/taniarzac-brookes/COMP5047

Introduction

This project forms part of the COMP5047 module and the structured development of multi-subsystem computer applications within a team. The application in question was to be developed for the United Student Unions federation. The system is designed to be capable of supporting the procedures of several universities' student unions across the country. The following sections include an analysis of quality requirements for a specific functional requirement, a use case model outlining the functional scope of the Student Union Management System (SUMS)

1 Project Management

Team structure

For this project our team will consist of four members: myself (Malachai Broderick), Ben Crosfield, Lucas Oliveira, and Tania Reyes. In our initial meeting, I took the liberty of collecting the other members' contact details and making a WhatsApp group, in which we would be able to communicate seamlessly for the rest of the project. As for the leadership of our group, Tania happily volunteered for the role of group leader, meaning our team's structure was established swiftly. This meant being responsible for monitoring progress checks and representing the team in discussions with the module leader. All other team members contribute equally to both technical and administrative tasks to promote fairness and prevent burnout within the over the course of this project.

Division of work and Subsystem assignment

Each team member will be responsible for and focused on the development of one subsystem These were also delegated in our first meeting. Listed below are the specific subsystems we will each be responsible for:

- USU Student App – Tania Reyes
- Student Union Management System – Malachai Broderick
- USU Operation System – Ben Crosfield
- Society Leader – Lucas Oliveira

All components, including agendas, meeting minutes, design documents, and UML models are stored in a GitHub repository accessible to all members of our team. Version control allows us to manage the branches for subsystems, merge work effectively, and track all of our contributions to efficiently integrate our project.

The team's work process is Agile based, involving weekly reviews and incremental development. Each week we review our deliverable against the coursework specification to assure we are meeting its requirements and fulfilling learning objectives.

2 Analysis and Specify Software Quality Requirements

Overview of Functional Requirement

FR-UO-5: This functional requirement, as listed in the relevant case study for this coursework project, outlines a key functional requirement of this subsystem. It goes on to mention that the system should enable unions to organise events, including initial set-up and updates on event progress.

For the Student Union Management Subsystem (SUMS), I will be focusing on the functional requirement based on management of Union events. This function grants union officers the ability of a specific university's student union to create, update, and promote events through this subsystem's web interface.

Each event record must include data defined in the case study (name, theme, date and time, venue, plan, participation constraints, scope, promotion scope, and promotion materials).

After an event has been registered, it is stored in the USU cloud backend and shared with other subsystems.

Subsystem Functionality

The subsystem I am looking at is a web interface used directly by the user. As such, it must have the required functionality to allow for user interaction (e.g. registration/creation of events, event management), effective communication with the backend or other subsystems in general.

Listed below are its specific functions:

- Allows each University's Student Union committee members/officers to register their login information, storing it in the backend for re-login.
- Societies can be created, modified, and terminated with approval from their respective student unions.
- Societies and unions can create, promote, and manage event
- All updates (e.g. event modification) are shared across all subsystems and synchronised in real-time.
- The site should be capable of handling ticket sales, allowing SU members to release event tickets at the desired quantity and price.

- Union officers must be able to approve or reject new society applications.
- Approval from the Union should notify the applicant through communication with the Society Leader App.
- Rejections will record the reason and mark the application as closed.

Quality Requirements

Society Management

Security & Privacy Protection:

- Only authenticated officers or society leaders will access society management functions via a role-based access control system, ensuring that only users with sufficient privileges will be able to approve or modify that society's records.
- Modification logs shall be timestamped, including an IP address and Student ID, being kept for 12 months to support more extensive auditing.
- Approval decisions classed as 'sensitive' (e.g. society termination) shall trigger multi-factor confirmation steps, preventing unauthorised or accidental actions.
- All file upload that relates to new societies shall be validated and virus scanned to prevent injection of malicious content into the cloud.
-

Performance:

- Update transactions will complete within _ seconds. Page loads listing all union societies will occur within 4 seconds for 95% of requests (server-side processing time).
- Page listing societies should load within 2 seconds for 95% of requests under normal conditions.
- Frequently accessed data will be pre-cached by the system to improve response times and reduce database load.
-

Reliability:

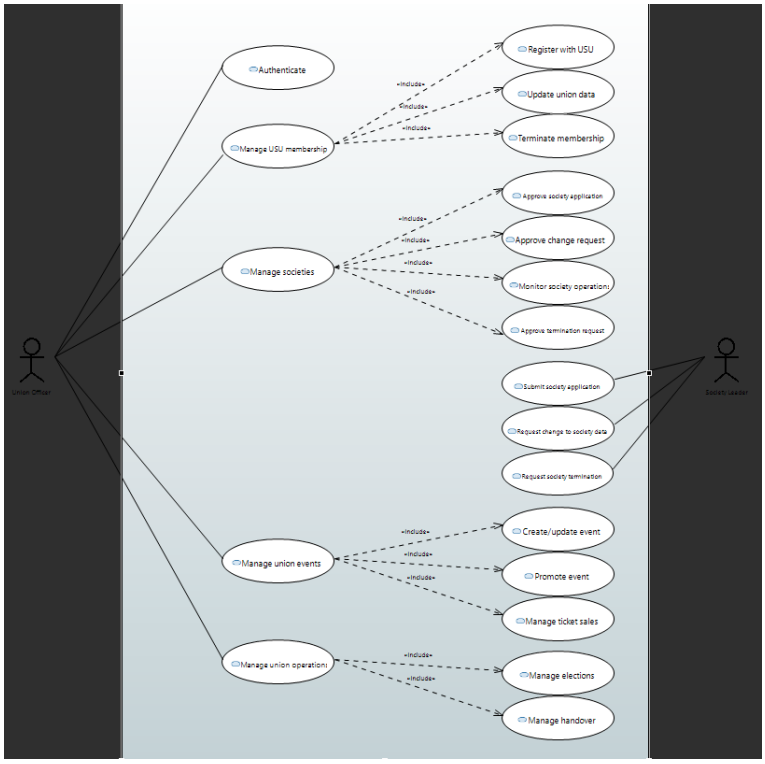
- If the society creation process fails mid transaction, the system will automatically roll back to the previous consistent state.
- Society management data will be replicated across multiple cloud availability zones to increase fault tolerance.
- The system will queue pending society update requests and automatically retry them following service outages.

Scalability:

- The database shall support up to 5000 individual societies per university.
- The system will support concurrent operations for hundreds of society leaders and officers during peak usage periods.
- The system architecture will support the onboarding of new universities, without the need for downtime or restructuring.

3 Specification and Software Functional Requirements

Use Case Model:

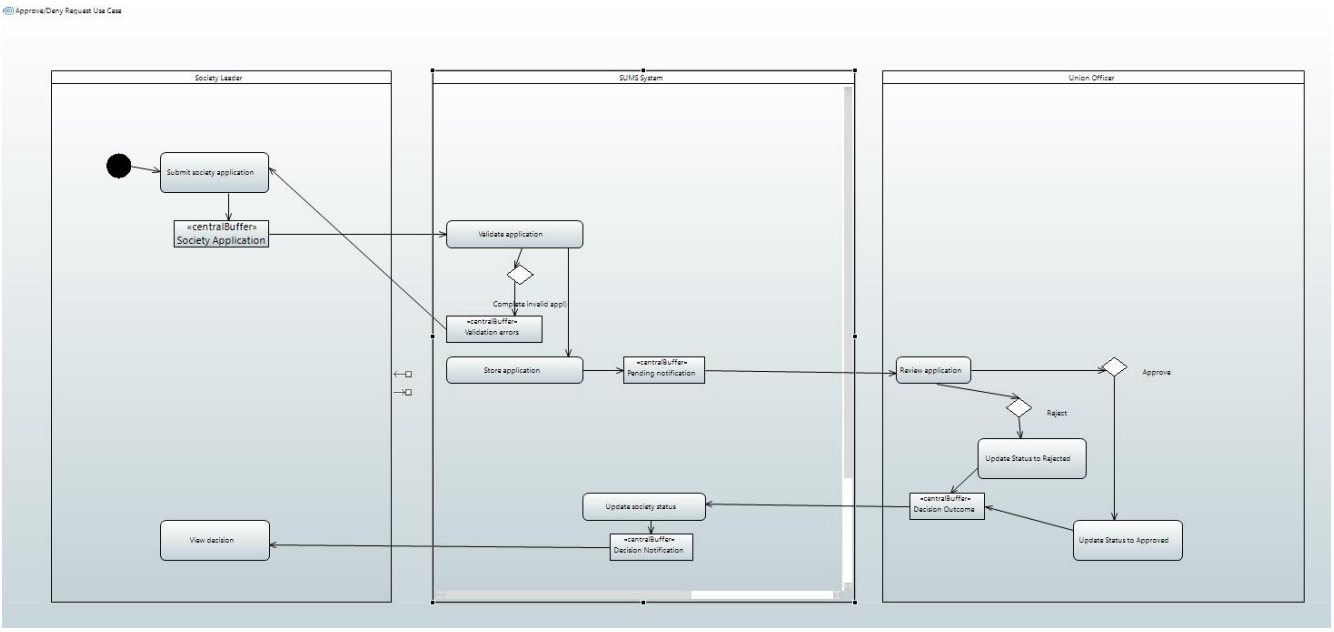


The use case model describes the functional scope of the Student Union Management System (SUMS). The primary actor is the **Union Officer**, a staff member of the university student union who accesses the system via the SUMS web interface. Authenticate must be completed before any other interaction is possible.

Manage events and Manage societies represent the two key activity domains of the system. Manage ticket sales is linked to Manage events via an «include» dependency, as arranging ticket availability and pricing is a mandatory step whenever an event is created or updated. Approve application and Reject application are both included by Manage societies, as a society submission always results in one of these two outcomes.

No extend relationships are modelled at this level of abstraction; all included use cases represent mandatory, unconditional behaviour rather than optional extensions.

Activity Diagram:



The activity diagram models the Approve/Reject Society Application workflow, selected as it demonstrates the most significant multi-step, branching interaction in SUMS and directly corresponds to the Manage societies use case (FR-UO-3). Three swim lanes partition responsibilities: the Society Leader Lane (left), the SUMS System Lane (centre), and the Union Officer Lane (right). Every element is placed inside the lane of the actor responsible for it.

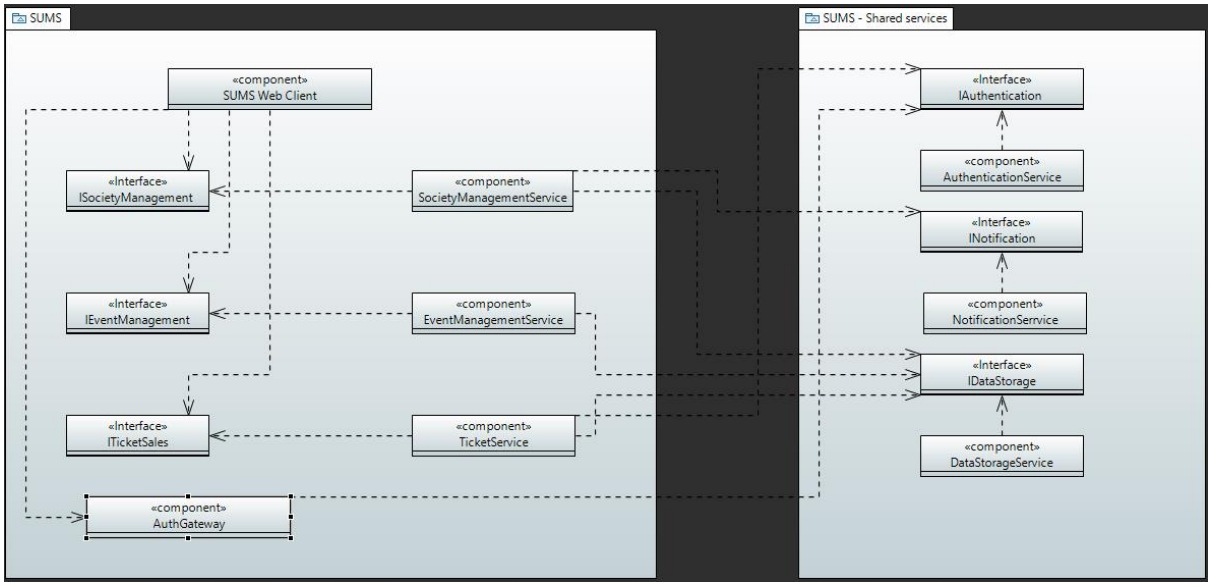
The workflow begins when the Society Leader applies, producing a central Buffer object Application data within their lane. This data crosses into the SUMS System Lane, where the system validates the submission. A decision node branches on whether the data is complete. If incomplete, SUMS produces a central Buffer Validation errors object in its lane and passes it back to the Society Leader, who corrects and resubmits the application via a loop back to the initial action. If complete, SUMS stores the application and produces a central Buffer Pending notification in its lane, which crosses into the Union Officer Lane.

The Union Officer reviews the application and reaches a second decision node. If the application is approved, control bypasses the rejection action. If rejected, the officer enters a rejection reason. Both branches converge at a merge diamond. The Union Officer then produces a central Buffer Decision outcome in their lane, which crosses back into the SUMS System Lane. SUMS updates the society status in the database and produces a central Buffer Decision notification, which is passed back to the Society Leader Lane where the final action View decision completes the workflow.

The diagram uses central Buffer nodes at every lane crossing to represent data artefacts in transit between actors. Decision nodes carry guard conditions in square brackets and are paired with a separate merge node where branches rejoin. The initial node is a filled circle, and the final node is a bullseye, in accordance with UML activity diagram syntax.

4 Software Architectural Design

Architecture of SUMS Subsystem:



The SUMS system is made up of small parts that work together. The SUMS Web Client is like the face of the system that people see when they use a web browser. It sends requests from Union Officers to the service in the back.

There are three services that handle the important work. The Society Management Service takes care of creating and changing societies and processing applications. The Event Management Service handles everything to do with events. The Ticket Service is, in charge of ticket sales and pricing. All three of these services need to use the Data Storage Service to work. The Society Management Service also needs to use the Notification Service to send alerts to the Society Leader App when something happens with an application.

When we look at the system, we can see lines that show how the different parts work together. These lines show how the client uses each service directly. There are also dashed lines that show which services need to work with shared services. These shared services are not part of the system, but they are used by the whole system.

Component	Stereotype	Provided Interface	Required Interfaces
SUMS Web Client	component	—	ISocietyManagement, IEventManagement, ITicketSales
AuthGateway	component	—	IAuthentication
SocietyMgmtService	service	ISocietyManagement	IDataStorage, INotification
EventMgmtService	service	IEventManagement	IDataStorage
TicketService	service	ITicketSales	IDataStorage
AuthenticationService	service	IAuthentication	—
NotificationService	service	INotification	—
DataStorageService	service	IDataStorage	—

The whole system is made up of all four USU subsystems that work together as one platform. Each USU subsystem has its special job and oversees its own area of work. It is like each USU subsystem has its box and does not share names with other USU subsystems.

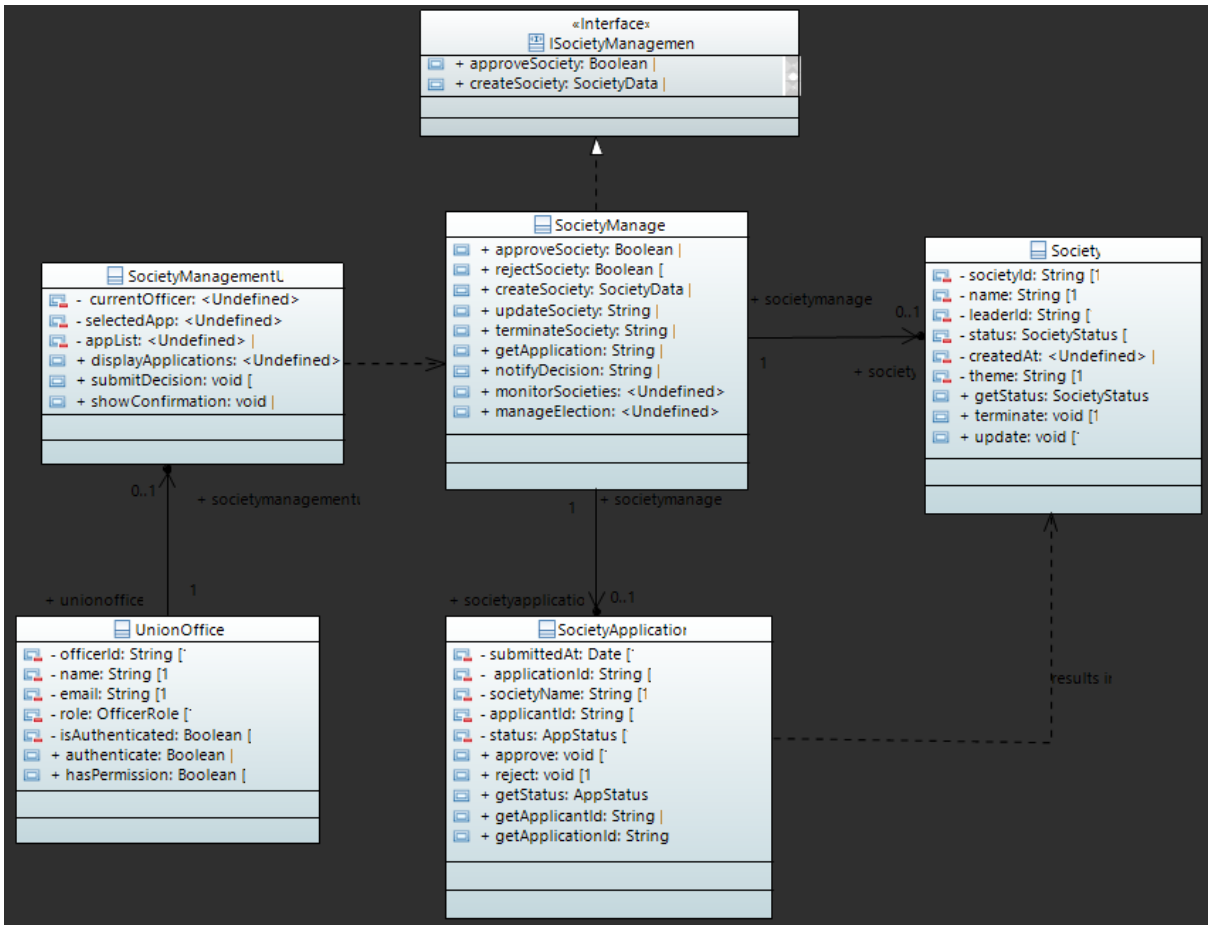
There are three services that all USU subsystems use. These are the Authentication Service, the Notification Service and the DataStorageService. They help make sure that everything works the way across the whole system. This way we do not have to repeat the work in each USU subsystem.

The USU Student App uses the Authentication Service and the DataStorageService. It uses these to help students sign up find events and buy tickets. The Society Leader App also uses the Notification Service. This is so it can get updates about society applications from SUMS. The SUMS subsystem uses all three services. It uses the Authentication Service to help officers log in. It uses the Notification Service to send out updates when something is approved or not approved. It uses the DataStorageService to store information about societies and events. The USU Operation System uses the Authentication Service and the DataStorageService to manage USU accounts.

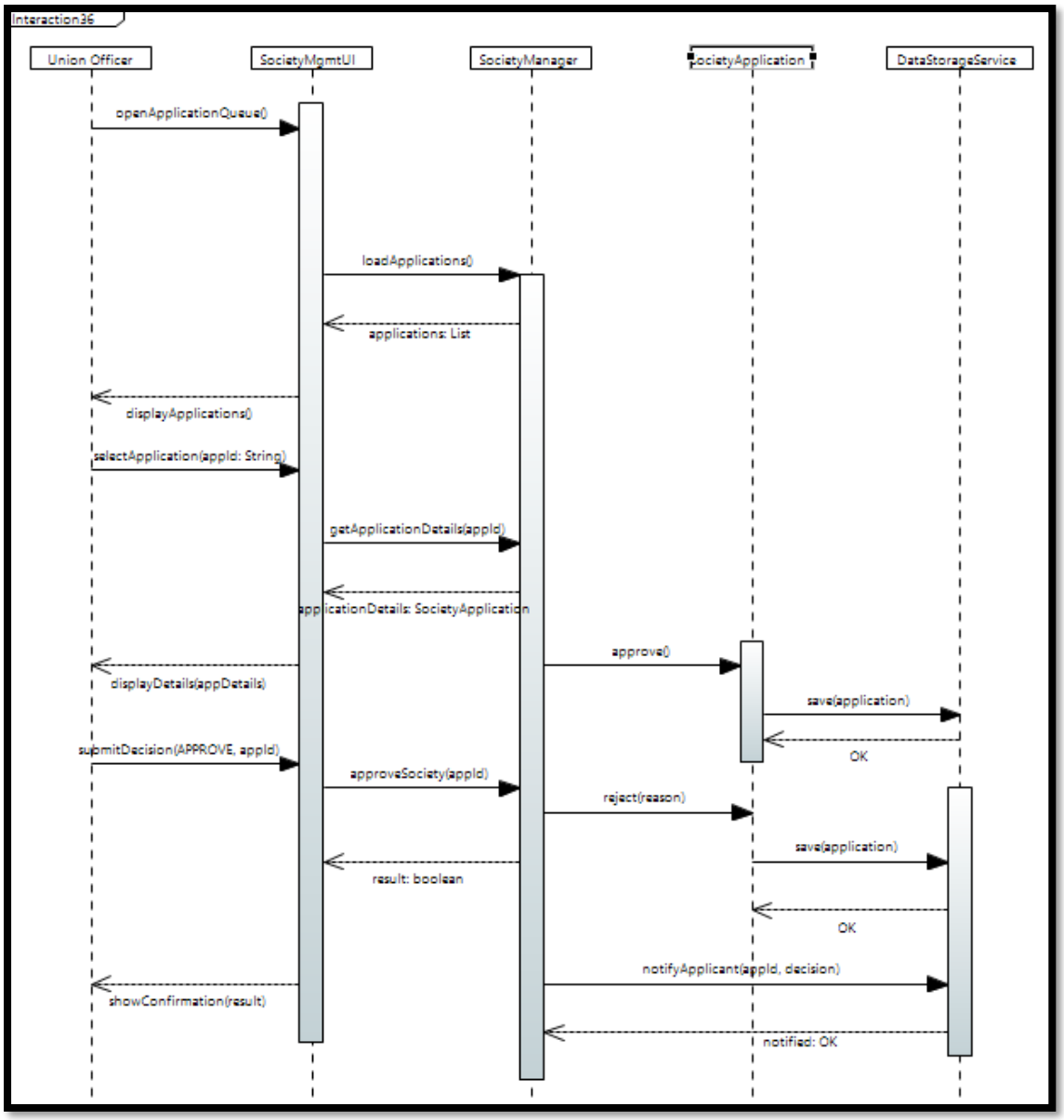
All USU subsystems work together through the services. This means that each USU subsystem is independent and does not affect the others directly. No USU subsystem has a link, to another USU subsystems internal parts.

5 Software Detailed Design

Structural Model:



Sequence Diagram:



The SocietyMgmtService component is designed using the Boundary-Control-Entity pattern. This pattern is used to organize the component. The ISocietyManagement interface is like a contract that the component shows to the rest of the system. Society Manager is the class that makes this contract real.

SocietyManagementUI is the class that handles what the officers see and do. It shows them the applications that are waiting gets their decisions and tells them what happened. It knows about the UnionOfficer and sends all the important work to the control class.

Society Manager is the control class. It does the main things: approve a society reject a society create a society update a society and end a society. It. Manages the Society and Society Application things.

Society is a class that holds the information about a registered society. It has a status that says what is happening with it. Society Application is a class that represents an application that is being worked on. It has a status and methods to approve or reject it. UnionOfficer is a class that holds the officer's information and has a method to check if they can do something.

All the attributes and methods are marked to say if they are private or public and what type they are.

The sequence diagram shows how the Approve or Reject Society Application works. It has five parts: the Union Officer, SocietyManagementUI Society Manager, Society Application and DataStorageService.

When the officer starts the application queue the boundary calls the control to get the applications. Then the officer looks at each one. The control gets its details. The officer decides what to do. The control does it. If the officer approves the control calls the method, on the Society Application. If the officer rejects the control calls the reject method.

After the officer decides the control tells the boundary what happened and the boundary shows it to the officer. Then the control tells the DataStorageService to notify the applicant.

The diagram uses frames to show what is happening. It has loops and conditions to make it clear what is going on. Each part of the diagram shows when it is working.